COMPUTE ACCELERATION FOR AI

CUDA on Kaggle

Hello World — Step-by-Step Lab Guide

A complete walkthrough: create your Kaggle account, verify your phone, enable GPU acceleration, and run your first CUDA kernel.

PREREQUISITES	DURATION	PLATFORM
Email address Phone number Internet access	~20 minutes First-time setup	Kaggle Notebooks Free tier GPU

LEARNING OBJECTIVES

- ✓ Create and configure a Kaggle account
- ✓ Verify your phone number to unlock GPU access
- ✓ Enable GPU acceleration in a Kaggle notebook
- ✓ Write, compile, and run a CUDA Hello World kernel
- ✓ Understand the basics of thread and block indexing

SECTION 1 · ACCOUNT SETUP

Creating Your Kaggle Account

STEP 1

Go to kaggle.com

Open your browser and navigate to **https://www.kaggle.com**. You will land on the Kaggle homepage.

> ■ **Recommended browsers**
>
> Use Google Chrome or Firefox for the best compatibility with Kaggle Notebooks.

STEP 2

Click 'Register'

In the top-right corner, click the **Register** button. You will see two options:

Option A	**Sign up with Google** — click the Google button and follow the OAuth flow. Recommended if you have a Google account.
Option B	**Register with email** — fill in your name, email, and a password, then click **Register**.

STEP 3

Confirm your email

If you registered with email, Kaggle will send a confirmation to your inbox. Open the email and click the **Verify Email** link.

> ■ **Email not arriving?**
>
> Check your Spam or Promotions folder. The email is sent from do-not-reply@kaggle.com.

STEP 4

Complete your profile

After email verification, Kaggle may ask you to fill in your username and select your experience level. Fill in the required fields and click **Next**.

Your username will be your public Kaggle identity — choose something professional, as it appears on your public notebooks and competition submissions.

SECTION 2 · PHONE VERIFICATION

Verifying Your Phone Number

■■ Why is phone verification required?

Kaggle requires a verified phone number to enable GPU and TPU accelerators. Without this step, the GPU option will be grayed out in your notebook settings.

STEP 1

Open your Account Settings

Click on your **profile picture** (or initials) in the top-right corner of any Kaggle page. In the dropdown, select **Settings**.

STEP 2

Find the Phone Verification section

Scroll down in Settings until you see the **Phone Verification** section. Click the **Add Phone Number** button.

```
Settings → scroll down → Phone Verification → Add Phone Number
```

STEP 3

Enter your phone number

Select your **country code** from the dropdown, then enter your **mobile phone number**. Click **Send Code**.

■ Important

Use a real mobile number that can receive SMS messages. VoIP numbers (Google Voice, Skype) are not accepted by Kaggle.

STEP 4

Enter the verification code

Kaggle will send a **6-digit SMS code** to your phone. Enter it in the field that appears and click **Verify**.

*The code expires after a few minutes. If it doesn't arrive within 60 seconds, request a new one by clicking **Resend Code**.*

STEP 5

Confirm GPU access is unlocked

Once verified, you should see a **green checkmark** next to Phone Verification in Settings. You now have access to free GPU quota:

GPU T4 x2	30 hours / week of free accelerator time
GPU P100	Available depending on region and remaining quota

SECTION 3 · FIRST CUDA KERNEL

Running Hello World on the GPU

STEP 1

Create a new Kaggle Notebook

From the Kaggle sidebar, click **Create** → **New Notebook**. A new Python notebook will open in your browser.

STEP 2

Enable the GPU Accelerator

In the right-hand panel, find **Session options** (the ■ icon). Under **Accelerator**, select **GPU T4 x2** and click **Save**. The notebook kernel will restart automatically.

```
Right panel → ■ Session options → Accelerator → GPU T4 x2 → Save
```

■■ GPU option grayed out?

Go back to Section 2 and complete phone verification first. Without verification, GPU accelerators are not available.

STEP 3

Verify the GPU is active

In the first notebook cell, type the command below and press **Shift + Enter** to run it:

```
!nvidia-smi
```

Expected output (abbreviated):

```
+-----------------------------------------------------------------------+
| NVIDIA-SMI 525.xx Driver Version: 525.xx CUDA Version: 12.x |
| GPU Name Temp Perf Pwr:Usage/Cap Memory-Usage GPU-Util |
| 0 Tesla T4 ... |
+-----------------------------------------------------------------------+
```

*If you see **CUDA Version: 12.x** and a GPU model name, you are ready to proceed.*

STEP 4

Check the CUDA compiler (nvcc)

In a new cell, verify that the NVIDIA CUDA compiler driver is installed:

```
!nvcc --version
```

Expected output:

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2022 NVIDIA Corporation
Cuda compilation tools, release 12.x, V12.x.xxx
```

STEP 5

Write the CUDA source file

In a new cell, use the **%%writefile** magic command to create a **.cu** source file. Copy the code exactly as shown:

```
%%writefile hello_cuda.cu
#include <stdio.h>
// __global__ marks this function as a GPU kernel
__global__ void helloFromGPU(void) {
printf("Hello from GPU! Thread %d, Block %d\n",
threadIdx.x, blockIdx.x);
}
int main(void) {
printf("Hello from CPU!\n");
// Launch: 2 blocks, 5 threads per block = 10 parallel threads
helloFromGPU<<<2, 5>>>();
// Block CPU until all GPU threads finish
cudaDeviceSynchronize();
printf("Done!\n");
return 0;
}
```

■ What does %%writefile do?

It is a Jupyter magic command that saves the cell content as a file on disk. After running this cell you will have hello_cuda.cu in your notebook directory.

STEP 6

Compile with nvcc

In a new cell, compile the .cu file using the NVIDIA CUDA compiler:

```
!nvcc hello_cuda.cu -o hello_cuda
```

If there are no errors, the cell completes silently and a binary file called **hello_cuda** will be created in the notebook directory.

■■ Compilation error?

Double-check that Step 5 output said "Writing hello_cuda.cu". Re-run that cell first, then try compiling again.

STEP 7

Execute the binary

Run the compiled binary in a new cell:

```
!./hello_cuda
```

You should see output similar to:

```
Hello from CPU!
Hello from GPU! Thread 0, Block 0
Hello from GPU! Thread 1, Block 0
Hello from GPU! Thread 2, Block 0
Hello from GPU! Thread 3, Block 0
Hello from GPU! Thread 4, Block 0
Hello from GPU! Thread 0, Block 1
Hello from GPU! Thread 1, Block 1
Hello from GPU! Thread 2, Block 1
Hello from GPU! Thread 3, Block 1
Hello from GPU! Thread 4, Block 1
Done!
```

■ Why is the thread order non-deterministic?

GPU threads execute in parallel. Their scheduling order is managed by the GPU hardware and may vary between runs — this is normal and expected behavior.

SECTION 4 · CODE BREAKDOWN

Understanding the Hello World Kernel

Keyword / Expression	Meaning
`__global__`	Marks a function as a GPU kernel. Called from CPU code, executed on the GPU by every launched thread.
`<<<2, 5>>>`	Kernel launch configuration: 2 blocks × 5 threads = 10 parallel threads running simultaneously.
`threadIdx.x`	Index of the current thread within its block (0 – 4).
`blockIdx.x`	Index of the current block within the grid (0 – 1).
`cudaDeviceSynchronize()`	Blocks the CPU until all GPU threads have completed. Required here to flush printf output from the GPU.

THREAD LAYOUT (<<<2, 5>>>)

Grid	
Block 0	T0 T1 T2 T3 T4

Block 1	T0 T1 T2 T3 T4

WHAT YOU ACCOMPLISHED

✓ Created a Kaggle account and verified your email
✓ Verified your phone number to unlock GPU acceleration
✓ Enabled a T4 GPU in a Kaggle Notebook
✓ Wrote a CUDA kernel using __global__, threadIdx.x and blockIdx.x
✓ Compiled with nvcc and executed 10 parallel GPU threads

■ YOUR EXERCISE

Now that Hello World is running, your task is to implement **GPU vector addition**. Write a CUDA kernel that takes two integer arrays **A** and **B**, and stores the element-wise sum in array **C**. Launch the kernel with one thread per element.

Hints: allocate memory on the GPU with cudaMalloc, copy data with cudaMemcpy, and use threadIdx.x to index into the arrays.